

Version Control

NCSC101

Prof. Pranay Kumar Saha

October 6, 2025





What is Version Control?

- A system that records changes to files over time
- Allows you to recall specific versions later
- Tracks **who** made changes, **what** was changed, and **when**
- Essential tool for software development and beyond
- Also known as: Source Control, Revision Control



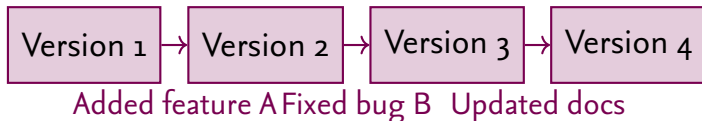
The Problem Without Version Control

Common Scenarios

- Multiple file versions: `project_final.doc`, `project_final_v2.doc`, `project_final_FINAL.doc`
- Lost work due to accidental deletions or overwrites
- Difficulty collaborating with team members
- No way to revert to previous working versions
- Unclear who made which changes



Version Control: Basic Concept



Each version represents a snapshot of your project at a specific point in time.



Why Version Control Matters

1. **Collaboration** - Multiple people can work simultaneously
2. **Backup** - Complete history of your project
3. **Experimentation** - Try new features without risk
4. **Accountability** - Track who made what changes
5. **Efficiency** - Streamlined workflow for teams



Collaboration Benefits

Team Development

- Multiple developers can work on different features simultaneously
- Automatic merging of changes when possible
- Conflict resolution when team members edit the same code
- Remote collaboration - work from anywhere
- Code review processes built into workflow



Backup and Recovery

Safety Net

- Complete history of every change ever made
- Revert entire project or individual files to any previous state
- Recover from mistakes quickly and easily
- Protection against data loss
- No more **Ctrl+Z** dependency!



Experimentation: Branching

- Create separate **branches** for experimental features
- Main codebase remains stable
- Test new ideas without affecting production code
- Merge successful experiments back into main branch
- Discard failed experiments without consequences

“Try, fail, learn, succeed - all without breaking production!”



Accountability and Tracking

Complete Audit Trail

- Who made each change?
- When was it made?
- Why was it made? (commit messages)
- What exactly changed? (line-by-line diffs)
- Useful for debugging, compliance, and learning



Beyond Software Development

Use Cases:

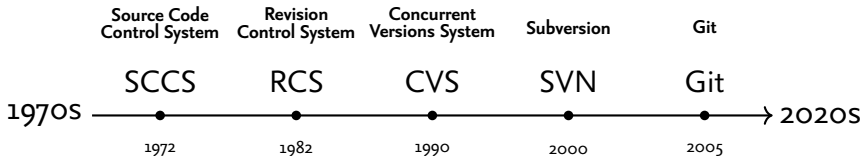
- Documentation
- Research papers
- Legal documents
- Configuration files
- Design files
- Data science projects

Benefits Apply:

- Track revisions
- Collaborate with co-authors
- Maintain versions
- Review changes
- Backup content



Version Control: Evolution Timeline



From local file-based systems to distributed version control



Before Version Control (Pre-1970s)

Manual Version Control

- Manual copying of files into backup directories
- Naming conventions: **v1**, **v2**, **final1**, **final2**
- Physical printouts of code
- Punch cards with version numbers written on them
- Error-prone and time-consuming
- Limited collaboration capability



SCCS (1972)

Source Code Control System

- Developed by Marc Rochkind at Bell Labs
- First version control system
- Single user, local file system
- Stored deltas (changes) instead of full copies
- Used on Unix systems
- Introduced concept of “check-in” and “check-out”

Limitation

Only one person could work on a file at a time (file locking)



RCS (1982)

Revision Control System

- Created by Walter F. Tichy at Purdue University
- Improved upon SCCS
- Faster and more efficient
- Better branching support
- Free and open-source
- Still single-user, single-machine
- Used reverse deltas for efficiency

RCS remained popular through the 1990s for individual developers



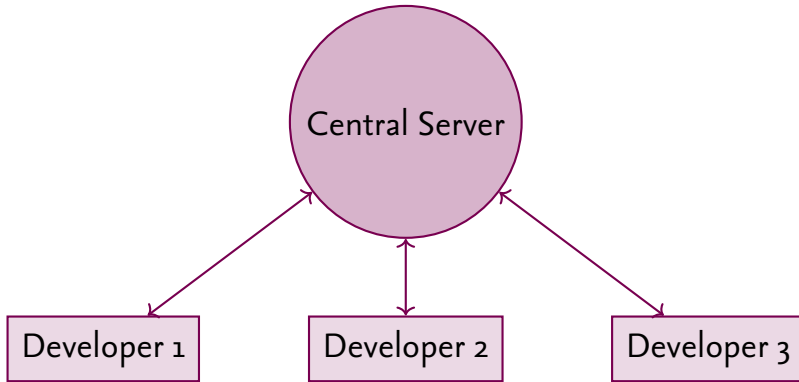
CVS (1990)

Concurrent Versions System

- Developed by Dick Grune, enhanced by many others
- First widely-used **centralized** system
- Multiple developers could work simultaneously
- Central repository model
- Network support (client-server architecture)
- Introduced concept of concurrent editing
- Revolutionized collaborative development



CVS: Centralized Model



All developers sync with central repository



Subversion - SVN (2000)

“CVS Done Right”

- Designed to fix CVS shortcomings
- Better handling of binary files
- Atomic commits (all or nothing)
- Efficient branching and tagging
- Directory versioning
- Still centralized like CVS
- Backed by Apache Software Foundation
- Widely adopted in enterprise environments



The Distributed Revolution (Mid-2000s)

Why Distributed?

Problems with centralized systems:

- Single point of failure (server down = no work)
- Slow operations over network
- Limited offline capabilities
- Branching/merging difficulties
- Bottleneck in large projects

Solution: Distributed Version Control Systems (DVCS)



Git (2005)

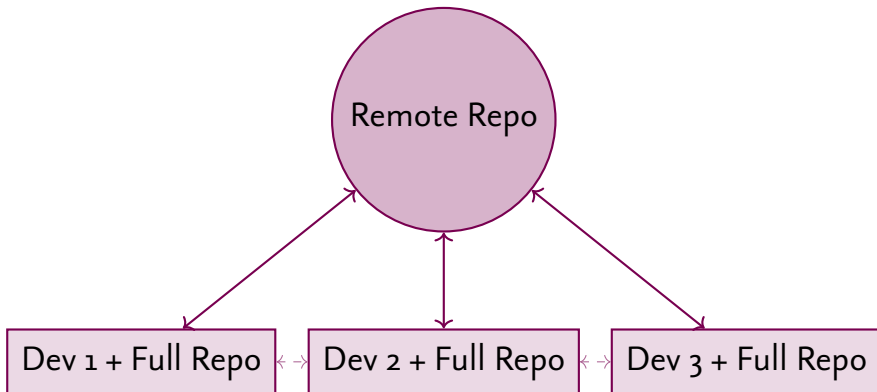
- Created by Linus Torvalds for Linux kernel development
- Revolutionary distributed architecture
- Every developer has complete repository history
- Extremely fast operations
- Powerful branching and merging
- Cryptographic integrity (SHA-1 hashing)
- Became the dominant version control system

Impact

Git transformed how software development teams collaborate worldwide



Distributed Version Control Model



Each developer has complete history



Distributed Version Control Model

- **Complete Repository Cloning**
 - Each developer has a full copy of the entire repository
 - Includes complete project history, all branches, and all commits
 - No dependency on central server for most operations
- **Offline Capabilities**
 - Commit, branch, merge, and view history without network access
 - Push and pull changes only when network is available
 - Developers can work independently and asynchronously



Distributed Version Control Model

- **Peer-to-Peer Collaboration**
 - Developers can share changes directly with each other
 - No strict hierarchy - flexible workflow models
 - Remote repository acts as coordination point, not bottleneck
- **Enhanced Backup and Redundancy**
 - Multiple complete copies provide natural disaster recovery
 - Loss of central server doesn't mean loss of project history



Other Distributed Systems

Mercurial (2005)

- Similar to Git
- Simpler interface
- Used by Facebook, Mozilla
- Still actively maintained

Bazaar (2005)

- Sponsored by Canonical
- Flexible workflows
- Used for Ubuntu development
- Less popular today

Git emerged as the clear winner in the DVCS wars



Modern Era: Hosting Platforms

Git Hosting Services (2008-present)

- **GitHub** (2008) - Largest platform, Microsoft-owned
- **GitLab** (2011) - Open-source, self-hostable
- **Bitbucket** (2008) - Atlassian, enterprise-focused

Added Features

- Web-based interfaces
- Issue tracking
- Code review tools (Pull Requests)
- CI/CD integration
- Project management
- Social coding features



Web-Based Interfaces

- **Browser-Based Repository Management**
 - Access and browse code from anywhere without local setup
 - Visual representation of repository structure and file contents
 - Syntax highlighting and code navigation in the browser
- **Simplified Collaboration**
 - Share repository links for quick code reviews
 - Edit files directly in the browser for quick fixes
 - View commit history, branches, and diffs visually
- **Documentation and Wiki Integration**
 - README files rendered with formatting
 - Built-in wiki for project documentation
 - Markdown support for rich text formatting



Issue Tracking

- **Centralized Bug and Task Management**
 - Create, assign, and prioritize issues within the repository
 - Track bugs, feature requests, and enhancements in one place
 - Link issues directly to commits and pull requests
- **Collaboration Features**
 - Comment threads for discussion and clarification
 - Tag team members with @mentions for notifications
 - Labels, milestones, and projects for organization
- **Transparency and Accountability**
 - Public issue tracking for open-source projects
 - Complete history of issue discussions and resolutions
 - Integration with development workflow



Code Review Tools (Pull Requests)

- **Structured Code Review Process**

- Propose changes through pull/merge requests
- Review code changes before merging to main branch
- Line-by-line commenting and discussion

- **Quality Assurance**

- Catch bugs and issues before they reach production
- Ensure code meets team standards and conventions
- Knowledge sharing through review comments

- **Approval Workflow**

- Require reviews from specific team members
- Automated checks before merging (tests, linting)
- Protected branches prevent unauthorized changes



CI/CD Integration

- **Continuous Integration (CI)**
 - Automatically build and test code on every commit
 - Run unit tests, integration tests, and code quality checks
 - Immediate feedback on breaking changes
- **Continuous Deployment (CD)**
 - Automatically deploy code to staging or production
 - Streamlined release process with minimal manual intervention
 - Rollback capabilities for failed deployments
- **Platform Integration**
 - Native CI/CD tools (GitHub Actions, GitLab CI, Bitbucket Pipelines)
 - Integration with external services (Jenkins, Travis CI, CircleCI)
 - Status badges showing build and test results



Social Coding Features

- **Community Engagement**
 - Star/favorite repositories to show appreciation
 - Follow developers and organizations
 - Discover trending projects and developers
- **Collaboration Across Boundaries**
 - Fork repositories to experiment and contribute
 - Contribute to open-source projects worldwide
 - Build portfolio showcasing contributions
- **Knowledge Sharing**
 - Public profiles displaying contributions and activity
 - Gists for sharing code snippets and scripts
 - Discussions and Q&A forums within repositories



Lessons from Version Control History

1. **Collaboration drives innovation** - Need for teamwork led to better tools
2. **Decentralization wins** - Distributed systems overcome centralized limitations
3. **Performance matters** - Fast operations increase adoption
4. **Open source accelerates progress** - Git, Linux kernel, community-driven
5. **Standards emerge naturally** - Market chose Git through merit



Summary

Importance

- Essential for collaboration, backup, and experimentation
- Provides accountability and project history
- Used beyond just software development

Evolution

- From local (SCCS, RCS) to centralized (CVS, SVN)
- From centralized to distributed (Git, Mercurial)
- Modern platforms add collaboration layers



Version Control

Thank You for Listening!